Web Engineering & Development (SWE 363)

Back-End Development: Node.js & Express II

In today's lecture:

- Query string parameters
- Posting form data
- Route parameters
- Route parameter middleware

Reference:

- Zybook: 6.5

Activity: Warm-up from last lecture (2 min)

```
const logRequest = function(req, res, next) {
  console.log(`Request: ${req.method} for ${req.path}`);
  next();
};
app.use(logRequest);
app.use(express.static("public"));
app.post("/goodbye", function(req, res) {
  res.send("<h1>Goodbye, Express!</h1>");
});
```

1. Which route matches a GET request to `/goodbye`?
2. What do the parameters `req`, `res`, and `next` represent?
3. If `next()` is removed, what is logged for `GET /goodbye`?
4. To be served, must `hello.html` be in `public` or in `node_modules`?

6.5 Express Request Data

How do we get data from the client?

Data	Example request	Read it from
Query string	`GET /hello?name=Bob&age=21`	`req.query`
Posted form data	`POST /hello` with body `name=Tamika&age=24`	`req.body`
Route parameter	`GET /users/jblack`	`req.params`

Each one is an object: `req.query.name`, `req.body.name`, `req.params.username`

What are Query String Parameters?

> A **query string** = `name=value` pairs after the `?` in a URL, separated by `&`

```
http://localhost:3000/hello?name=Bob&age=21
```

- Path → `/hello`
- Query string → `name=Bob&age=21`
- Express **automatically parses** it into the `req.query` object
- Forms with `method="get"` send their data this way

Reading Query Parameters in Express

```
// GET http://localhost:3000/hello?name=Bob&age=21
app.get("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.query.name}!</h1>
    <p>You are ${req.query.age} years old.</p>`;
  res.send(html);
});
```

Browser shows: Hello, Bob! You are 21 years old.

- The query string doesn't change the route: `/hello` still matches

Activity: Predict what the page shows

```
app.get("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.query.name}!</h1>
    <p>You are ${req.query.age} years old.</p>`;
  res.send(html);
});
```

1. `/hello?name=Sara&age=20`
2. `/hello?age=20`
3. `/hello?name=Doctor+Who&age=900` (in a URL, `+` encodes a space)
4. `/hello?city=Dallas` : is the city in `res.query.city` or `req.query.city` ?

Posting Form Data

> A form that **posts** to an Express route sends its data **URL-encoded** in the **body**

public/hello.html and the **HTTP request** it sends on Submit:

```
<form method="post" action="/hello">
  <p><label>Name? <input type="text" name="name"></label></p>
  <p><label>Age? <input type="number" name="age"></label></p>
  <input type="submit" value="Submit">
</form>
```

```
POST /hello HTTP/1.1
Content-Type: application/x-www-form-urlencoded

name=Tamika&age=24
```

Receiving URL-Encoded Data

```
const express = require("express");
const app = express();

app.use(express.static("public"));                  // serves hello.html
app.use(express.urlencoded({extended: false}));  // parses the form data

app.post("/hello", function(req, res) {
  const html = `<h1>Hello, ${req.body.name}!</h1>
    <p>You are ${req.body.age} years old.</p>`;
  res.send(html);
});

app.listen(3000);
```

- `express.urlencoded()` → **middleware** that adds the parsed values to **`req.body`**
- Only parses bodies with `Content-Type: application/x-www-form-urlencoded`

Activity: Put the steps in order

A. The middleware parses the body and attaches the data to `req.body`

B. The browser requests `hello.html`, and `express.static()` serves it

C. The `/hello` POST route sends back "Hello, Tamika!"

D. The user types Tamika and 24 and presses Submit

E. The browser sends `POST /hello` with `name=Tamika&age=24` in the body

Activity: What happens if... (each change on its own)

```
<!-- public/hello.html -->
<form method="post" action="/hello">
  <input type="text" name="name">
  <input type="number" name="age">
  <input type="submit" value="Submit">
</form>
```

```
app.use(express.static("public"));
app.use(express.urlencoded({extended: false}));

app.post("/hello", function(req, res) {
  res.send(`<h1>Hello, ${req.body.name}!</h1>`);
});
```

1. Submit is clicked with no name?
2. `express.urlencoded()` is removed?
3. The form uses `method="get"`?
4. The request's Content-Type is `application/json` instead of `application/x-www-form-urlencoded`?
5. `app.use(express.static("public"))` is removed?

What are Route Parameters?

> A **route parameter** = a value near or at the end of the URL path; its name is defined with a **colon** (`:`)

```
// GET http://localhost:3000/users/jblack
app.get("/users/:username", function(req, res) {
  const username = req.params.username;   // jblack
  res.send("<h1>Profile for " + username + "</h1>");
});
```

- Route parameters are attached to the `req.params` object
- One route → a **dynamic webpage** for every username

Browser shows: Profile for jblack

Activity:

Which req property (`req.query` , `req.params` , or `req.body`)? What value?

1. Route `/users/:username` , request `GET /users/abc`
2. Route `/search` , request `GET /search?term=node`
3. Route `app.post("/major", ...)` , a form posts `major=SWE`
4. Route `/city/:cityName` , `GET /city/Denver` : is it `req.params.city` ?
5. Route `/users/:username` , request `GET /users/` : does the page say "Profile for "?

Route Parameter Middleware

> `app.param()` defines **parameter middleware** that executes **before** a route's callback function

- The middleware function has a **fourth parameter** → the route parameter's value
- It can add data to `req` for the route to use, then calls `next()`

```
GET /users/jblack
  → app.param("username", ...)          sets req.user, calls next()
  → app.get("/users/:username", ...)    uses req.user
  → Profile for Jack Black
```

A real application would look up the user in a **database** (next class)

Param Middleware Example

```
// Parameter middleware executes before the route
app.param("username", function(req, res, next, username) {
  // Check if username is recognized
  if (username === "jblack") {
    req.user = { name: "Jack Black", username: username };
  }
  else {
    req.user = { name: "Unknown", username: username };
  }
  // Continue processing the request
  next();
});

app.get("/users/:username", function(req, res) {
  // req.user contains the user info set in the parameter middleware
  res.send("<h1>Profile for " + req.user.name + "</h1>");
});
```

Sending JSON Responses

```
app.get("/users/:username", function(req, res) {
  if (req.user.name !== "Unknown") {
    res.json(req.user);
  }
  else {
    res.status(404).json({ error: "Unknown user" });
  }
});
```

- `res.json()` → encodes a JavaScript object as **JSON** (`application/json`)
- `res.status(404)` → sets the **status code** (404 = Not Found) before sending
- `/users/jblack` → `{ "name": "Jack Black", "username": "jblack" }`
- `/users/sara` → 404 and `{ "error": "Unknown user" }`

Activity: Predict the response

```
app.param("zip", function(req, res, next, zip) {
  if (zip === "80015") {
    req.forecast = {high: 95, low: 72};
  }
  next();
});
app.get("/weather/:zip", function(req, res) {
  if (req.forecast) { res.json(req.forecast); }
  else { res.status(404).json({ error: "Unknown ZIP code" }); }
});
```

1. `/weather/80015`
2. `/weather/12345` : what status code?
3. `next()` is removed: `/weather/80015` ?
4. `app.param("zipcode", ...)` instead of `"zip"` : `/weather/80015` ?

Activity: Exit ticket: find the 4 bugs

```
app.param("courseId", function(req, res, next, courseId) {
  req.course = { code: courseId, title: "Web Engineering" };
});
app.get("/courses/:code", function(req, res) {
  res.json(req.course);
});
// GET /search?term=node
app.get("/search", function(req, res) {
  res.send("Searching for " + req.params.term);
});
// A form posts major=SWE (no other app.use() calls)
app.post("/major", function(req, res) {
  res.send("Major: " + req.body.major);
});
```

Key Takeaways

- **Query string** (`?name=Bob&age=21`) → `req.query`
- **Posted form data** → `express.urlencoded()` parses it into `req.body`
- **Route parameter** (`/users/:username`) → `req.params`
- `app.param()` runs **before** the route's callback and must call `next()`
- `res.json()` sends JSON; `res.status(404).json()` also sets the status code

30m

Demo

6.4 Request Data in Express

- Query string parameters
- Route parameters
- Param middleware
- JSON responses with status codes

Next Class

- Mongoose (7.1)
- MongoDB (7.2)